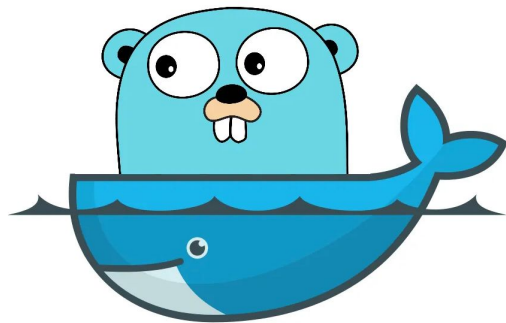




Beyond unit tests: integration testing

Go microservices with Docker

Go service + real Postgres, Redis, Kafka in containers

Repo: <https://github.com/2beens/go-integration-testing>

Today's Agenda

- Integration testing: the space between **unit** and **E2E**
- **Demo app** - example of a service handling payments
- A **Go** service that uses **Kafka**, **Redis**, **Postgres** and **HTTP**
- How to **run**, **debug**, **break** and **ship** integration tests in CI



Where Integration Tests Fit

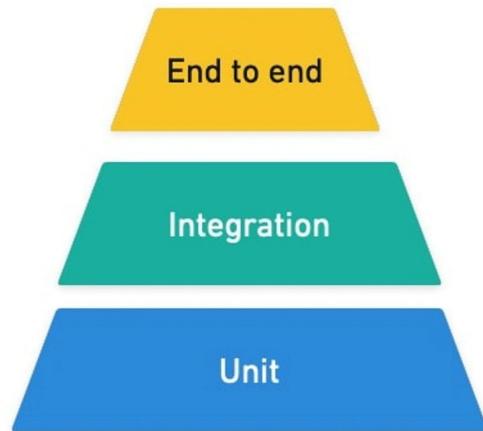
Unit tests - fast, isolated, mock-heavy

Integration tests - real dependencies, balanced speed and confidence

E2E tests - realistic, slow, expensive

- We usually have the two ends - unit and E2E
- The useful middle is often missing

The Test Pyramid



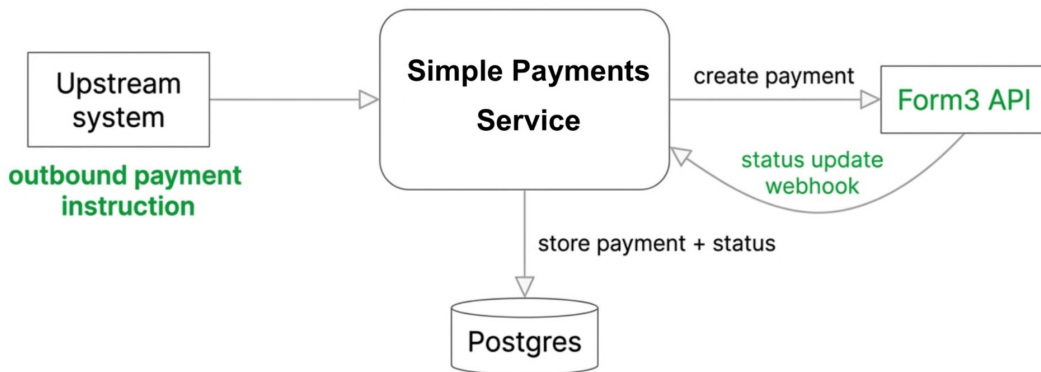
Benefits

- **Production-like startup path**
- **Real protocols and dependencies:** Postgres, Redis, Kafka, HTTP
- **Catches wiring mistakes** unit tests often miss
- **Easy to debug** — breakpoints in app + test with `go test` command
- Same test command locally and in the **CI**

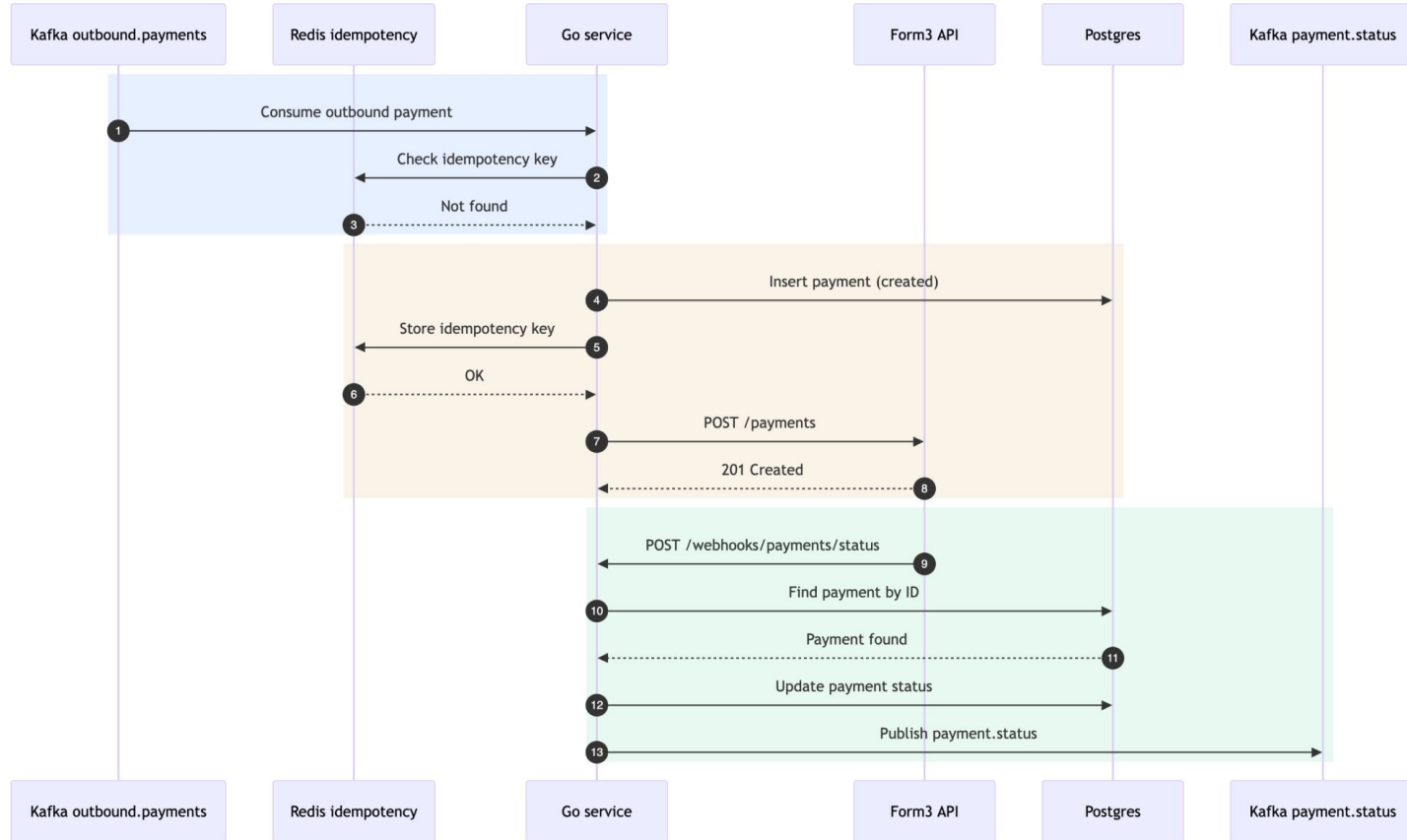
The payment flow we test

As a demo, we're using a small Go service for outbound payments.

1. Kafka receives **outbound.payment**
2. Redis checks and stores idempotency key
3. Service calls Form3 API
4. Store payment in Postgres
5. Webhook updates payment status
6. Service emits **payment.status**



The payment flow we test (happy path)



Demo - integration package

internal/integration/

- **setup_test.go** — bootstrap app + containers
- **payments_test.go** — main integration scenarios
- **kafka_test.go** — Kafka publish / consume helper functions
- **form3_mock_test.go** — Form3 HTTP mock + request recorder for assertions

We're using the **testcontainers/testcontainers-go** to create and clean up container-based dependencies.

Demo - run the tests

- Gated by `RUN_INTEGRATION_TESTS=1`
- Easy to run when we need them
- Same command locally and in CI

```
RUN_INTEGRATION_TESTS=1 go test -v  
./internal/integration/...
```

Demo - break bootstrap / init

- Temporarily break  something:
 - wrong config keys/values, or
 - consumer not started, or
 - DB migrations not started, etc.
- Run unit tests: **still green** 
- Run integration tests: **fails** with a clear error 

Point: integration guards **composition**, not just functions.

Demo - 2 integration tests

TestOutboundPayment_HappyPath

1. **Publish** `outbound.payments` → wait until the payment exists in the DB
2. **Verify side effects**: the service calls Form3 API with the right payload and stores the idempotency key in Redis
3. **Simulate** a `completed` webhook, then assert the DB is updated, a `payment.status` event is emitted, and `GET /payments/{id}` returns the final state

TestOutboundPayment_Duplicate

1. **Publish** first `outbound.payment` → wait for **one** payment + **one** Form3 API call
2. **Publish duplicates** (same idempotency key)
3. Assert still **one** payment, **same** ID, **no extra** Form3 API calls

Demo - Debugging in VS Code

- Use existing VS Code launch config: [Debug Integration Tests](#)
- Put breakpoints in test or app code
- Step through startup, Kafka, Redis, DB, and HTTP flow

```
{  
  "name": "Debug integration test",  
  "type": "go",  
  "request": "launch",  
  "mode": "test",  
  "env": { "RUN_INTEGRATION_TESTS": "1" }  
}
```

Demo - From local runs to CI

- Separate integration-tests job in CI
- GitHub runners already have Docker
- No extra CI services: testcontainers starts Postgres, Redis, and Kafka
- Reuse the same test setup as local development

```
- name: Run integration tests
  env:
    RUN_INTEGRATION_TESTS: "1"
  run: go test -v -timeout=120s ./internal/integration/...
```

When this works well

 use it for:

- critical flows you really do not want to break
- flows where Postgres, Redis, Kafka, and HTTP must work together
- verifying real payloads, events, and wiring

 watch out for:

- flakiness from async behavior and timing
- slower feedback locally and in CI
- docker/image availability and external image pulls
- maintenance cost if this layer grows too much

Takeaway: integration tests protect composition; they complement unit tests, not replace them 

